Assignment No : 01

Course Code : CSM 2122
Course Title : Data Structure

Bangladesh Agricultural University , Mymensingh-2202

DATE OF ASSIGNMENT		
17	03	2026

DATE OF SUBMISSION		
03	04	2026

SUBMITTED TO:	SUBMITTED BY:
Mr. Md. Saif Uddin **Lecturer** **Department of Computer Science** **Faculty of Agricultural Engineering & Technology**	**Name : MD.MONTASIR HOSSEN TANVIR** **Level : 2** **Semester : 1** **ID No : 2409020** **Reg No : 58984** **Session : 2023-2024** **B.Sc in Bioinformatics Engineering**

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1

PART A

Array Based Problem

P1 Second Largest and Second Smallest (Without Sorting)

1. Problem Statement

Write a program to find the **second largest** and **second smallest** elements in an array **without sorting the array**. Also analyze the **time complexity** of your solution and mention whether any **better or more optimized approach** is possible.

2. Algorithm

1. Start
2. Read array size n
3. Read array elements
4. Initialize:
   - largest = smallest = arr[0]
5. Traverse array to find **largest** and **smallest**
6. Initialize:
   - secondLargest = INT_MIN
   - secondSmallest = INT_MAX
7. Traverse array again:
   - If element > smallest AND element < secondSmallest → update secondSmallest
   - If element < largest AND element > secondLargest → update secondLargest
8. Print second largest and second smallest
9. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/main.cpp

main.cpp

```cpp
#include <bits/stdc++.h>

using namespace std;

int main()
{
    int n;

    cout << "Enter the number of elements : ";
    cin >> n;

    int arr[n];

    cout<<"Enter array elements : ";
    for(int i=0; i<n; i++){
        cin >> arr[i];
    }
    int largest = arr[0], smallest = arr[0];

    for(int i = 1; i < n; i++) {
        if(arr[i] > largest)
            largest = arr[i];
        if(arr[i] < smallest)
            smallest = arr[i];
    }

    int secondLargest = INT_MIN;
    int secondSmallest = INT_MAX;

    for(int i = 0; i < n; i++) {
        if(arr[i] > secondLargest && arr[i] < largest)
            secondLargest = arr[i];

        if(arr[i] < secondSmallest && arr[i] > smallest)
            secondSmallest = arr[i];
    }

    cout << "Second largest elements : "<<secondLargest<<endl;
    cout << "Second smallest elements : "<<secondSmallest<<endl;

    return 0;
}
```

4. Sample Input and Output

```
Enter the number of elements : 5
Enter array elements : 1 2 7 -10 100
Second largest elements : 7
Second smallest elements : 1

Process returned 0 (0x0)   execution time : 12.457 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
Best	O(N)	O(1)
Average	O(N)	O(1)
Worst	O(N)	O(1)

P2 Count Frequency of Each Distinct Element

1. Problem Statement

Write a program to **count the frequency of each distinct element** in an array.

2. Algorithm

1. Start
2. Read size n
3. Read array elements
4. Create array frequency[256] initialized to 0
5. For each element i from 0 to n-1
    - Set i=0
    - freequency[arr[i]] = freequency[arr[i]] ++
6. For each element i from 0 to 256
    - Set i=0
    - If frequency[arr[i]] > 0
    - Print element and its frequency
7. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-2.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){

  int freequency[256] = {0};

  int n;

  cout<<"Enter number of elements: ";
  cin >> n;

  int arr[n];

  cout<<"Enter elements: ";
  for(int i=0; i<n; i++){
     cin >> arr[i];
     freequency[arr[i]]++;
  }
  for(int i=0; i<256; i++){
     if(freequency[i] > 0 ){
        cout <<i << "-> Frequency: " << freequency[i] << endl;
     }
  }


return 0;
}
```

4. Sample Input and Output

```
Enter number of elements: 5
Enter elements: 1 2 3 2 10
1-> Frequency: 1
2-> Frequency: 2
3-> Frequency: 1
10-> Frequency: 1

Process returned 0 (0x0)   execution time : 18.188 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
Best	O(N)	O(N)
Average	O(N)	O(N)
Worst	O(N)	O(N)

P3 Reverse an Array In-Place

1. Problem Statement

Reverse an array in-place without using any additional array. The operation must be done with O(1) extra space.

2. Algorithm

1. Start
2. Read size n
3. Read array elements
4. Initialize two variables:
    - start = 0
    - end = n - 1
5. While start < end
    - Swap arr[start] and arr[end]
    - Increment start
    - Decrement end
6. Print reversed array
7. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-3.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){
  int n;
  cout<<"Enter number of elements: ";
  cin >> n;
  int arr[n];

  cout <<"Enter elements: ";
  for(int i=0; i<n; i++){
     cin >> arr[i];
  }
    int start = 0;
    int last = n - 1;

    while(start < last) {
        int temp = arr[start];
        arr[start] = arr[last];
        arr[last] = temp;

        start++;
        last--;
    }
    cout << "Reversed array: ";
    for(int i=0; i<n; i++){
         cout << arr[i] << " ";
    }
return 0;
}
```

4. Sample Input and Output

```
Enter number of elements: 6
Enter elements: 1 2 3 9 8 7
Reversed array: 7 8 9 3 2 1
Process returned 0 (0x0)   execution time : 10.855 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
All Cases	O(N/2) = O(N)	O(1)

P4 Merge Two Arrays and Remove Duplicates

1. Problem Statement

Write a program to **merge two arrays into one** and **remove duplicate elements** from the resulting array.

2. Algorithm

1. Start
2. Declare an integer array freeq[256] and initialize all elements to 0.
3. Input size of first array n1.
4. Input n1 elements into array arr1.
5. Input size of second array n2.
6. Input n2 elements into array arr2.
7. Create a new array merged of size n1 + n2.
8. Set variable len = 0.
9. Copy first array into merged array
    - For i = 0 to n1 - 1
    - merged[len] = arr1[i]
    - len = len + 1
10. Copy second array into merged array
    - For i = 0 to n2 - 1
    - merged[len] = arr2[i]
    - len = len + 1

11. Remove duplicates using frequency array

- For i = 0 to n1 + n2 - 1
- If freeq[merged[i]] == 0
- Print merged[i]
- Set freeq[merged[i]] = 1

13. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-4.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){

    int n1,n2,freeq[256]={0};
    cout <<"Enter size of first array: ";
    cin >> n1;

    int arr1[n1];
    cout << "Enter elements: ";
    for(int i=0; i<n1; i++){
        cin >> arr1[i];
    }

    cout << "Enter size of second array: ";
    cin >> n2;

    int arr2[n2];
    cout << "Enter elements: ";
    for(int i=0; i<n2; i++){
        cin >> arr2[i];
    }

    int merged[n1 + n2];
    int len = 0;

    for(int i = 0; i < n1; i++) {
        merged[len++] = arr1[i];
    }

    for(int i = 0; i < n2; i++) {
        merged[len++] = arr2[i];
    }

    cout << "Merged without duplicates: ";
    for(int i=0; i<(n1+n2); i++){
        if(freeq[merged[i]] == 0){
            cout << merged[i] <<" ";
            freeq[merged[i]] = 1;
        }
    }

return 0;
}
```

4. Sample Input and Output

```
Enter size of first array: 4
Enter elements: 1 2 9 20
Enter size of second array: 4
Enter elements: 20 20 3 8
Merged without duplicates: 1 2 9 20 3 8
Process returned 0 (0x0)   execution time : 16.632 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
Worst	O(n1+n2)	O(n1+n2)

P5 Rotate Array to the Left by k Positions

1. Problem Statement

Given an array of N elements, rotate the array to the left by k positions. Elements shifted off the left end wrap around to the right.

2. Algorithm

1. Start
2. Read size of array n
3. Read array elements
4. Read number of positions k
5. For i = 0 to k-1:
    - Store first element in temp
    - Shift all elements left by one
    - Place temp at the last position
6. Print rotated array
7. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-5.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){

 int n,k;
 cout << "Enter number of elements : ";
 cin >> n;

 int arr[n];
 cout << "Enter elements: ";
 for(int i=0; i<n; i++){
      cin >> arr[i];
 }
 cout << "Enter k: ";
 cin >> k;

      k = k % n;
    for(int i = 0; i < k; i++) {
        int temp = arr[0];
        for(int j = 0; j < n - 1; j++) {
            arr[j] = arr[j + 1];
        }
        arr[n - 1] = temp;
    }
    cout << "Array after left rotation by "<<k<<" : ";
    for(int i=0; i<n; i++){
         cout << arr[i] <<" ";
    }
return 0;
}
```

4. Sample Input and Output

```
Enter number of elements : 7
Enter elements: 9 1 8 5 3 2 10
Enter k: 1
Array after left rotation by 1 : 1 8 5 3 2 10 9
Process returned 0 (0x0)   execution time : 17.451 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
All Cases	O(N*k)	O(1)

P6 Check if Array Forms a Palindrome

1. Problem Statement

Write a program to **check whether the elements of an array form a palindrome**.
A palindrome array reads the same forward and backward. Also state the **time complexity**.

2. Pseudocode

1. Start
2. Read size of array n
3. Read array elements
4. Initialize two pointers:
   - start = 0
   - end = n - 1
5. While start < end:
   - If arr[start] != arr[end] → Not a palindrome, exit
   - Increment start, decrement end
6. If loop completes → array is a palindrome
7. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-6.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout <<"Enter elements: ";
    for(int i=0; i<n; i++){
         cin >> arr[i];
    }
    int start = 0, last = n - 1;
    int isPalindrome = 1;

    while(start < last) {
        if(arr[start] != arr[last]) {
            isPalindrome = 0;
            break;
        }
        start++;
        last--;
    }
    if(isPalindrome){
        cout <<"The array is a palindrome."<<endl;
    }
    else{
        cout << "The array is not a palindrome."<<endl;
    }
return 0;
}
```

4. Sample Input and Output

```
Enter number of elements: 5
Enter elements: 1 2 3 2 1
The array is a palindrome.

Process returned 0 (0x0)   execution time : 7.542 s
Press any key to continue.
```

```
Enter number of elements: 5
Enter elements: 2 4 3 2 4
The array is not a palindrome.

Process returned 0 (0x0)   execution time : 7.661 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
Best (early mismatch)	O(1)	O(1)
Average	O(N/2)= O(N)	O(1)
Worst (palindrome)	O(N/2)= O(N)	O(1)

P7 Delete All Occurrences of a Value (O(N))

1. Problem Statement

Write a program to **delete all occurrences of a specified value** from an array while **preserving the order** of the remaining elements. The algorithm must run in **O(N)** time complexity.

2. Algorithm

1. Start
2. Read array size n
3. Read array elements
4. Read value val to delete
5. Initialize index = 0 (position for next kept element)
6. For each element i from 0 to n-1:
   - If arr[i] != val → arr[index++] = arr[i]
7. New size of array → index
8. Print array elements from 0 to index-1
9. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-7.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){

    int n,val;
    cout << "Enter number of elements: ";
    cin >>n;

    int arr[n];
    cout << "Enter elements: ";
    for(int i=0; i<n; i++){
        cin >> arr[i];
    }
    cout << "Enter value to delete: ";
    cin >> val;

     int index = 0;
     for(int i = 0; i < n; i++) {
         if(arr[i] != val) {
             arr[index++] = arr[i];
         }
     }
     cout <<"Array after deleting all " <<val << " : ";
     for(int i=0; i<index; i++){
           cout << arr[i] <<" ";
     }

return 0;
}
```

4. Sample Input and Output

```
Enter number of elements: 5
Enter elements: 1 2 9 19 5
Enter value to delete: 19
Array after deleting all 19 : 1 2 9 5
Process returned 0 (0x0)   execution time : 15.134 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
All Cases	O(N)	O(1)

P8 Insert into a Sorted Array Preserving Order

1. Problem Statement

Write a program to **insert a new element into a sorted array** such that the **sorted order is preserved**. Analyze the **time complexity**.

2. Algorithm

1. Start
2. Read size n and elements of sorted array arr[n]
3. Read element x to insert
4. Initialize i = n - 1
5. While i >= 0 and arr[i] > x:
    - Shift arr[i] to arr[i+1]
    - Decrement i
6. Insert x at arr[i+1]
7. Increment array size → n = n + 1
8. Print updated array
9. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-8.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){
      int n,x;
      cout<<"Enter size of sorted array: ";
      cin >> n;
      int arr[n+1];
      cout << "Enter sorted elements: ";
      for(int i=0; i<n; i++){
          cin >> arr[i];
      }
      cout << "Enter element to insert: ";
      cin >> x;
      int i = n - 1;
      while(i >= 0 && arr[i] > x) {
         arr[i + 1] = arr[i];
         i--;}
      arr[i + 1] = x;
      n++;
      cout <<"Array after insertion: ";
      for(int j=0; j<n; j++){
          cout << arr[j] <<" ";
      }
return 0;
}
```

4. Sample Input and Output

```
Enter size of sorted array: 5
Enter sorted elements: 1 2 4 5 6
Enter element to insert: 3
Array after insertion: 1 2 3 4 5 6
Process returned 0 (0x0)   execution time : 8.878 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
Best (insert at end)	O(1)	O(1)
Average	O(N/2) = O(N)	O(1)
Worst (insert at start)	O(N)	O(1)

P9 Update All Occurrences of a Value (O(N), Single Pass)

1. Problem Statement

Write a program to **update all occurrences of a given value** in an array with another value. The program must **scan the array only once** and run in **O(N) time complexity**.

2. Algorithm

1. Start
2. Read size n
3. Read array elements arr[n]
4. Read oldValue (value to replace) and newValue (replacement)
5. For each element i = 0 to n-1:
   - If arr[i] == oldValue, set arr[i] = newValue
6. Print updated array
7. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-9.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){
    int n, oldValue, newValue;
    cout<<"Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout<<"Enter elements: ";
    for(int i=0; i<n; i++){
        cin >> arr[i];
    }
    cout << "Enter value to replace: ";
    cin >> oldValue;
    cout << "Enter new value:  ";
    cin >> newValue;

    for(int i = 0; i < n; i++) {
        if(arr[i] == oldValue) {
            arr[i] = newValue;
        }}
    cout << "Updated array: ";
    for(int i=0; i<n; i++){
        cout << arr[i] << " ";
    }
return 0;
```

4. Sample Input and Output

```
Enter number of elements: 5
Enter elements: 1 2 8 5 9
Enter value to replace: 9
Enter new value:  0
Updated array: 1 2 8 5 0
Process returned 0 (0x0)   execution time : 18.897 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
All Cases	O(N)	O(1)

P10 Remove Duplicates from Unsorted Array (Maintain First-Occurrence Order)

1. Problem Statement

Write a program to **remove duplicate elements** from an **unsorted array** while **maintaining the order** of the first occurrences. Explain the **time complexity** of the method used.

2. Algorithm

1. Start
2. Read size n and array elements arr[n]
3. Initialize index = 0 for storing unique elements
4. For each element i = 0 to n-1:
    - Check if arr[i] is already present in arr[0…index-1]
    - If not present → store arr[index] = arr[i], increment index
5. Print elements from arr[0…index-1]
6. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-10.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout<<"Enter elements: ";
    for(int i=0; i<n; i++){
        cin >> arr[i];
    }
    int index = 0;
     for(int i = 0; i < n; i++) {
        int found = 0;
        for(int j = 0; j < index; j++) {
            if(arr[i] == arr[j]) {
                found = 1;
                break;
            }}
        if(!found) {
            arr[index++] = arr[i];
        }}
   cout <<"Array after removing duplicates: ";
    for(int i=0; i<index; i++){
         cout << arr[i]<<" ";
    }
return 0;
}
```

4. Sample Input and Output

```
Enter number of elements: 7
Enter elements: 1 2 1 3 4 2 2
Array after removing duplicates: 1 2 3 4
Process returned 0 (0x0)   execution time : 37.969 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
Worst	$O(N^2)$	$O(N)$

P11 Compress Array by Removing Zeros (O(N))

1. Problem Statement

Write a program to **compress an array** by **removing all zero values** and **shifting the remaining elements to the left**. The operation must be completed in **O(N) time complexity**.

2. Algorithm

1. Start
2. Read size n and array elements arr[n]
3. Initialize index = 0 to track next non-zero element position
4. Traverse the array from i = 0 to n-1:
   - If arr[i] != 0 → arr[index++] = arr[i]
5. After traversal, remaining elements from index to n-1 can be ignored (or set to 0 if needed)
6. Print first index elements as compressed array
7. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-11.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){

  int n;
  cout << "Enter number of elements: ";
  cin >> n;

  int arr[n];
  cout << "Enter elements: ";
  for(int i=0; i<n; i++){
     cin >> arr[i];
  }
   int index = 0;
    for(int i = 0; i < n; i++) {
        if(arr[i] != 0) {
            arr[index++] = arr[i];
        }
    }

    cout << "Compressed array (zeros removed) : ";
    for(int i=0; i<index; i++){
        cout << arr[i] <<" ";
    }

return 0;
}
```

4. Sample Input and Output

```
Enter number of elements: 5
Enter elements: 1 0 9 0 0
Compressed array (zeros removed) : 1 9
Process returned 0 (0x0)   execution time : 10.369 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
All Cases	O(N)	O(1)

P12 Insert Element After Every Occurrence of a Target Value

1. Problem Statement

Write a program to **insert a new element immediately after every occurrence of a specified value** in an array.

2. Algorithm

1. Start
2. Read size n and array elements arr[n]
3. Read target (value after which to insert) and newValue (value to insert)
4. Initialize newSize = n
5. Traverse the array to **count occurrences of target** → to calculate new array size
6. Create a new array result[newSize] (or shift elements in-place from the end)
7. Traverse original array:
    - Copy element to result[]
    - If element == target → insert newValue next
8. Print updated array
9. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/Arry-12.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

int main(){
 int n, target, newValue;
 cout<<"Enter number of elements: ";
 cin >> n;

 int arr[n];
 cout << "Enter elements: ";
 for(int i=0; i<n; i++){
    cin >> arr[i];
 }
 cout <<"Enter target value: ";
 cin >> target;

 cout << "Enter value to insert after each occurrence: ";
 cin >> newValue;

 int i = 0;
    while(i < n) {
        if(arr[i] == target) {
            for(int j = n; j > i+1; j--) {
                arr[j] = arr[j-1];
            }
            arr[i+1] = newValue;
            n++;
            i++;
        }
        i++;
    }
  cout << "Resulting array: ";
   for(int i=0; i<n; i++){
     cout << arr[i] << " ";
   }
 return 0;
}
```

4. Sample Input and Output

```
Enter number of elements: 6
Enter elements: 1 2 3 2 4 2
Enter target value: 2
Enter value to insert after each occurrence: 99
Resulting array: 1 2 99 3 2 99 4 2 99
Process returned 0 (0x0)   execution time : 5.130 s
Press any key to continue.
```

5. Time Complexity Analysis

Case	Time Complexity	Space Complexity
Best	O(N)	O(N)
Average	O(N)	O(N)
Worst	O(N)	O(2N) = O(N)

PART B

Link List Based Problem

P1 Count Total Number of Nodes in a Singly Linked List.

1. Problem Statement

Write a program to count the total number of nodes in a singly linked list.

2. Algorithm

1. Start
2. Initialize count = 0
3. Traverse the linked list from head
4. For each node, increment count
5. Move to next node
6. Repeat until NULL
7. Print count
8. End

3. C++ Program Implementation

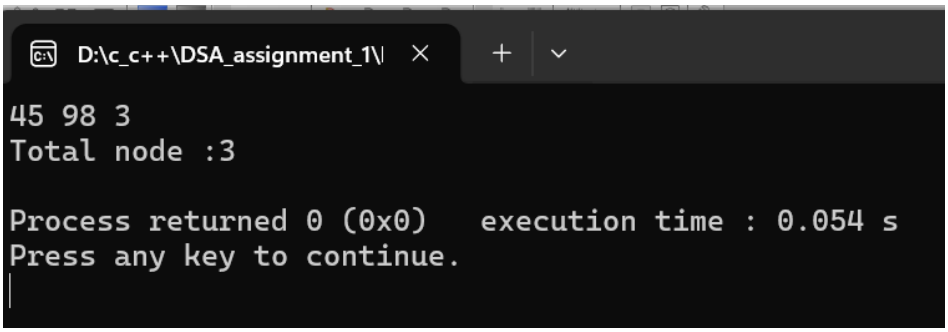
Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/ll-1.cpp

ll-1.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

struct node{
    int data;
    struct node *link;
};

void countNode(struct node *head){

    struct node *temp = head;

    int counT = 0;

    while(temp != NULL){
    cout << temp -> data <<" ";
    counT ++;
    temp = temp -> link;
 }

 cout <<endl<< "Total node :" << counT<<endl;
}

int main(){

```

```cpp
int main(){

 struct node *head,*current;

 head = (struct node *)malloc(sizeof(struct node));
 head -> data = 45;
 head -> link = NULL;

 current = (struct node *)malloc(sizeof(struct node));
 current -> data = 98;
 current -> link = NULL;

 head -> link = current;


 current = (struct node *)malloc(sizeof(struct node));
 current -> data = 3;
 current -> link = NULL;

 head -> link -> link = current;

  countNode(head);

return 0;
}
```

4. Sample Input and Output



5. Time Complexity Analysis

Traverse all nodes and increment a counter; **O(N)** time because every node is visited, **O(1)** space since no extra storage is used.

P2 Find the Middle Node of a Singly Linked List

1.Problem Statement :

Write a program to find the middle node of a singly linked list. If the list contains an even number of nodes, return the second middle node.

2. Algorithm

1. Start
2. Initialize two pointers: regular and fast pointing to head
3. Move regular by one step and fast by two steps
4. Repeat until fast reaches NULL or fast->next becomes NULL
5. regular pointer will point to the middle node
6. Print the middle node value
7. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/ll-2.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

    struct node{
    int data;
    struct node *link;
};

void findMiddle(struct node *head){
    struct node *regular = head, *fast = head;

    if(head == NULL){
        cout << "List is empty";
        return;
    }
    while(fast != NULL && fast->link != NULL){
            regular = regular -> link;
            fast = fast -> link -> link;
    }

    cout << regular -> data <<endl;
}

int main(){
        struct node *head,*current,*temp;
    int n;
    cout <<"Enter the number of elements : ";
    cin >> n;


    head = NULL;

    cout << "Enter the element : ";

    for(int i=0; i<n; i++){
        current = (struct node *)malloc(sizeof(struct node));
        cin >> current -> data;

        if(head == NULL){
            head = current;
        }
        else{
            temp -> link = current;
        }
        temp = current;
    }

    findMiddle(head);


return 0;
}
```

4. Sample Input and Output

```
D:\c_c++\DSA_assignment_1\

Enter the number of elements : 5
Enter the element : 1 2 3 4 5
3

Process returned 0 (0x0)   execution time : 7.802 s
Press any key to continue.
```

5.Time Complexity Analysis

Use slow and fast pointers to locate the middle; **O(N)** time as each node is touched once, **O(1)** space for two pointers.

P3 Reverse a Singly Linked List

1.Problem Statement :

Write a program to reverse a singly linked list without using any extra linked list.

2. Algorithm

1. Start
2. Initialize three pointers: prev = NULL, current = head, next = NULL
3. Repeat until current is NULL
4. Store next = current->next
5. Change current->next = prev
6. Move prev = current
7. Move current = next
8. Set head = prev
9. End

3. C++ Program Implementation

Source Codec: https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/ll-3.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;

struct node{
  int data;
  struct node *link;
  };

  struct node* revrse(struct node *head){

      struct node *current = head,*prev = NULL, *next = NULL;

      if(current == NULL){
        cout <<"List is empty" <<endl;
        return NULL;
      }

      while(current != NULL){
          next = current -> link;
          current -> link = prev;
          prev = current;
          current = next;
      }

      return prev;
  }

  void print(struct node *head){
     struct node *temp = head;

     while(temp != NULL){
        cout <<temp -> data <<" ";
        temp = temp -> link;
     }
     cout << endl;
  }
int main(){
    int n;
    cout <<"Enter the number of elements : ";
    cin >> n;

    struct node *head,*current,*temp;

    head = NULL;

    cout << "Enter the elements : ";
    for(int i=0; i<n; i++){
        current = (struct node *)malloc(sizeof(struct node));
        cin >> current -> data;

        if(head == NULL){
            head = current ;
        }
        else{
            temp -> link = current;
        }
        temp = current;
    }
      cout <<"Original list : ";
      print(head);

      head = revrse(head);
      cout << "Reverse : ";
      print(head);
      cout<<endl;
return 0;
}
```

4. Sample Input Output

```
D:\c_c++\DSA_assignment_1\

Enter the number of elements : 7
Enter the elements : 100 200 5999 700 1 9 10
Original list : 100 200 5999 700 1 9 10
Reverse : 10 9 1 700 5999 200 100


Process returned 0 (0x0)   execution time : 12.955 s
Press any key to continue.
```

5. Time Complexity Analysis

Reverse links in-place; **O(N)** time because each node's pointer is updated once, **O(1)** space as no extra list is needed.

P4 Delete All Occurrences of a Specified Value from a Singly Linked List

1.Problem Statement :

Write a program to delete all occurrences of a specified value from a singly linked list. The solution should preserve the order of the remaining nodes.

2. Algorithm Start

1. Initialize temp = head and prev = NULL
2. Traverse the list
3. If node data equals the specified value
   - If node is head → move head to next
   - Else link prev->next to temp->next
   - Delete the node
4. Else move prev forward
5. Move temp forward
6. Repeat until end
7. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/ll-4.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
struct node{
    int data;
    struct node *link;

    };
 struct node* deleteValue(struct node *head,int key){
        struct node *temp = head, *prev = NULL;
        if(head == NULL){
            cout <<"List is empty"<<endl;

            return NULL;
        }
     while(temp != NULL){
         if(temp->data == key){
             if(prev == NULL){
                 head = temp->link;
                 free(temp);
                 temp = head;
             }else{
                 prev->link = temp->link;
                 free(temp);
                 temp = prev->link;
             }
         }else{
             prev = temp;
             temp = temp->link;
         }
     }
     return head;
 }
   void print(struct node *head){
     struct node *temp = head;

     while(temp != NULL){
        cout <<temp -> data <<" ";
        temp = temp -> link;
     }
     cout << endl;
  }
int main(){
    int n;
     cout <<"Enter the number of elements : ";
     cin >> n;

     struct node *head,*current,*temp;
     head = NULL;
     cout << "Enter the elements : ";
     for(int i=0; i<n; i++){
         current = (struct node *)malloc(sizeof(struct node));
         cin >> current -> data;

         if(head == NULL){
             head = current ;
         }
         else{
             temp -> link = current;
         }
         temp = current;
     }
     cout << "Original list : ";
     print(head);
     int key;
     cout<<"Enter value that will delete : ";
     cin >> key;

     head = deleteValue(head,key);
     cout<<"After delete : ";
     print(head);
return 0;
}
```

4. Sample Input Output

```
D:\c_c++\DSA_assignment_1\
Enter the number of elements : 6
Enter the elements : 1 2 3 2 5 2
Original list : 1 2 3 2 5 2
Enter value that will delete : 2
After delete : 1 3 5

Process returned 0 (0x0)   execution time : 9.835 s
Press any key to continue.
```

5. Time Complexity Analysis

Traverse and unlink matching nodes; **O(N)** time to visit all nodes, **O(1)** space since changes are done in-place.

P5 Update All Occurrences of a Specified Value in a Singly Linked List

1.Problem Statement :

Write a program to update all occurrences of a specified value in a singly linked list with a new value. The solution must run in O(N) time complexity.

2. Algorithm

1. Start
2. Set temp = head
3. Traverse the linked list
4. If temp->data equals old value
5. Replace it with new value
6. Move to next node
7. Repeat until NULL
8. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/ll-5.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;
struct node{
  int data;
  struct node *link;
  };
 void update(struct node *head,int old,int newValue){
      struct node *temp = head;

      if(head == NULL){cout<<"List is empty";
      return;}

      while(temp != NULL ){
           if(temp -> data == old){
             temp -> data = newValue;
           }
           temp = temp -> link;
      }
 }
  void print(struct node *head){
     struct node *temp = head;

     while(temp != NULL){
        cout <<temp -> data <<" ";
        temp = temp -> link;
     }
     cout << endl;
  }
int main(){
    int n;
    cout <<"Enter the number of elements : ";
    cin >> n;
    struct node *head,*current,*temp;
    head = NULL;
    cout << "Enter the elements : ";
    for(int i=0; i<n; i++){
        current = (struct node *)malloc(sizeof(struct node));
        cin >> current -> data;

        if(head == NULL){
            head = current ;
        }
        else{
            temp -> link = current;
        }
        temp = current;
    }
    cout << "Original list : ";
    print(head);
    int old,newValue;
    cout<<"Old value : ";
    cin>>old;
    cout<<"New Value : ";
    cin >> newValue;
    update(head,old,newValue);
    cout<<"After update : ";
    print(head);
return 0;
}
```

4. Sample Input Output

```
D:\c_c++\DSA_assignment_1\

Enter the number of elements : 4
Enter the elements : 10 20 10 40
Original list : 10 20 10 40
Old value : 10
New Value : 99
After update : 99 20 99 40

Process returned 0 (0x0)   execution time : 36.933 s
Press any key to continue.
```

5. Time Complexity Analysis

Traverse and replace values; **O(N)** time for visiting every node, **O(1)** space as values are updated in-place.

P6 Insert a New Node After Every Occurrence of a Specified Value

1.Problem Statement :

Write a program to insert a new node after every occurrence of a specified value in a singly linked list.

2. Algorithm

1. Start
2. Set temp = head
3. Traverse the linked list
4. If temp->data equals specified value
5. Create a new node
6. Insert new node after temp
7. Move temp to next of inserted node
8. Else move temp to next node
9. Repeat until NULL
10. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/ll-6.cpp

```cpp
#include<bits/stdc++.h>

using namespace std;
struct node{
    int data;
    struct node *link;
    };
void insertAfter(struct node *head,int key,int newValue){
        struct node *temp = head;
        struct node *newNode;
        if(head == NULL){cout<<"List is empty";
        return;}
        while(temp != NULL){
            if(temp -> data == key){
              newNode = (struct node*)malloc(sizeof(struct node));
              newNode -> data = newValue;

              newNode -> link = temp -> link;
              temp -> link = newNode;

              temp = newNode -> link;
            }else{
              temp = temp -> link;}
        }
    }
    void print(struct node *head){
      struct node *temp = head;

      while(temp != NULL){
         cout <<temp -> data <<" ";
         temp = temp -> link;
      }
      cout << endl;
    }
int main(){
     int n;
     cout <<"Enter the number of elements : ";
     cin >> n;
     struct node *head,*current,*temp;
     head = NULL;
     cout << "Enter the elements : ";
     for(int i=0; i<n; i++){
         current = (struct node *)malloc(sizeof(struct node));
         cin >> current -> data;

         if(head == NULL){
             head = current ;
         }
         else{
             temp -> link = current;
         }
         temp = current;
     }
     cout << "Original list : ";
     print(head);
     int key,newValue;
     cout <<"Insert value after : ";
     cin>>key;
     cout<<"New Value : ";
     cin >>newValue;

     insertAfter(head,key,newValue);
     cout <<"After insert : ";
     print(head);
return 0;
}
```

4. Sample Input Output

```
D:\c_c++\DSA_assignment_1\

Enter the number of elements : 3
Enter the elements : 10 20 10
Original list : 10 20 10
Insert value after : 10
New Value : 99
After insert : 10 99 20 10 99

Process returned 0 (0x0)   execution time : 10.875 s
Press any key to continue.
```

5. Time Complexity Analysis

Traverse and add new nodes after matches; **O(N)** time to scan all nodes, **O(1)** extra space as new nodes don't count as extra memory.

P7 Remove Duplicate Nodes from an Unsorted Singly Linked List

1.Problem Statement :

Write a program to remove duplicate nodes from an unsorted singly linked list while keeping the first occurrence unchanged.

2. Algorithm

1. Start
2. Set pointer1 = head
3. While pointer1 is not NULL
4. Set pointer2 = pointer1
5. While pointer2->next is not NULL
6. If pointer1 data equals pointer2->next data
7. Delete duplicate node
8. Else move pointer2 forward
9. Move pointer1 forward

10. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/ll-7.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
struct node{
  int data;
  struct node *link;
  };
void removeDuplicates(struct node* head){
    struct node *ptr1, *ptr2, *dup;
    ptr1 = head;
    while(ptr1 != NULL && ptr1->link != NULL){
        ptr2 = ptr1;

        while(ptr2->link != NULL){
            if(ptr1->data == ptr2->link->data){
                dup = ptr2->link;
                ptr2->link = ptr2->link->link;
                free(dup);
            }else{
                ptr2 = ptr2->link;
            }
        }
        ptr1 = ptr1->link;
    }
}
  void print(struct node *head){
     struct node *temp = head;

     while(temp != NULL){
        cout <<temp -> data <<" ";
        temp = temp -> link;
     }
     cout << endl;
  }
int main(){
  int n;
    cout <<"Enter the number of elements : ";
    cin >> n;
    struct node *head,*current,*temp;
    head = NULL;
    cout << "Enter the elements : ";
    for(int i=0; i<n; i++){
        current = (struct node *)malloc(sizeof(struct node));
        cin >> current -> data;

        if(head == NULL){
            head = current ;
        }
        else{
            temp -> link = current;
        }
        temp = current;
    }
    cout << "Original list : ";
    print(head);
    removeDuplicates(head);
    cout<<"After delete duplicate : ";
    print(head);

return 0;}
```

4. Sample Input Output

```
D:\c_c++\DSA_assignment_1\

Enter the number of elements : 4
Enter the elements : 10 20 10 20
Original list : 10 20 10 20
After delete duplicate : 10 20

Process returned 0 (0x0)   execution time : 19.834 s
Press any key to continue.
```

5. Time Complexity Analysis

Check each node for duplicates; **O(N²)** without hash because each node compares with all others, **O(1)** space. With hash, **O(N)** time and **O(N)** space for the hash set.

P8 Check Whether a Singly Linked List is a Palindrome

1.Problem Statement :

Write a program to check whether a singly linked list is a palindrome.

2. Algorithm

1. Start
2. Initialize slow and fast pointers at head
3. Reverse first half of the list while finding middle
4. If number of nodes is odd, skip middle node
5. Compare reversed first half with second half
6. If all elements match → palindrome
7. Else → not palindrome
8. End

3. C++ Program Implementation

Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/ll-8.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

struct node{
    int data;
    struct node *link;
    };

    int isPalindrome(struct node* head){
     struct node *regular=head,*fast=head;
     struct node *prev=NULL,*temp=NULL;

     while(fast != NULL && fast->link != NULL){
         fast = fast->link->link;

         temp = regular->link;
         regular->link = prev;
         prev = regular;
         regular = temp;
     }
     if(fast != NULL){
         regular = regular->link;
     }

     while(prev != NULL&& regular != NULL){
         if(prev->data != regular->data)
             return 0;
         prev = prev->link;
         regular = regular->link;
     }
     return 1;
}

     void print(struct node *head){
      struct node *temp = head;

     while(temp != NULL){
         cout <<temp -> data <<" ";
         temp = temp -> link;
     }
     cout << endl;
 }

int main(){

     int n;
     cout <<"Enter the number of elements : ";
     cin >> n;
     struct node *head,*current,*temp;
     head = NULL;
     cout << "Enter the elements : ";
     for(int i=0; i<n; i++){
         current = (struct node *)malloc(sizeof(struct node));
         cin >> current -> data;

         if(head == NULL){
             head = current ;
         }
         else{
             temp -> link = current;
         }
         temp = current;
     }
     cout << "Original list : ";
     print(head);
     if(isPalindrome(head))
     {cout<<"The list is palindrome."<<endl;}
     else{
         cout<<"The list is not palindrome."<<endl;}
return 0;}
```

4. Sample Input Output

```
D:\c_c++\DSA_assignment_1\
Enter the number of elements : 4
Enter the elements : 10 20 20 10
Original list : 10 20 20 10
The list is palindrome.

Process returned 0 (0x0)   execution time : 24.637 s
Press any key to continue.
```

```
D:\c_c++\DSA_assignment_1\
Enter the number of elements : 5
Enter the elements : 10 1 5 100 0
Original list : 10 1 5 100 0
The list is not palindrome.

Process returned 0 (0x0)   execution time : 9.638 s
Press any key to continue.
```

5. Time Complexity Analysis

Reverse half list or use stack; **O(N)** time for traversal and comparison, **O(1)** in-place or **O(N)** for extra stack/array.

P9	Operations on Doubly, Circular, and Doubly Circular Linked Lists

1. Problem Statement:

Apply the following operations on Doubly, Circular, and Doubly Circular Linked Lists:

2. Algorithm & C++ Implementation
Source Code : https://github.com/Tanvirr0777/DSA_assignment_1/blob/main/ll-9-dl.cpp

(a) Inserting new node at –
• Beginning
• End
• At a specific location
• In a sorted list

- **At the Beginning**

1. Create a new node.
2. Set new node's data.
3. Set new node's prev to NULL.
4. Set new node's next to head.
5. If head is not NULL, set head->prev = new node.
6. Update head = new node.

```
void insertBeginning(struct dnode **head){
        int newValue;
        cout<<"Enter new value: ";
       cin >> newValue;
    struct dnode *newNode = (struct dnode*)malloc(sizeof(struct dnode));

    newNode->data = newValue;
    newNode->prev = NULL;
    newNode->link = *head;

    if(*head != NULL)
        (*head)->prev = newNode;

    *head = newNode;
     cout<<"Now list is : ";
     print(*head);
}
```

- **At the End**

1. Create a new node.
2. Set new node's data.
3. Traverse to the last node.
4. Set last node's next = new node.

5. Set new node's prev = last node.
6. Set new node's next = NULL.

```
void insertEnd(struct dnode **head){
    int newValue;
    cout<<"Enter new value: ";
    cin >> newValue;

    struct dnode *newNode = (struct dnode*) malloc(sizeof(struct dnode));
    newNode->data=newValue;
    newNode->link=NULL;

    struct dnode *temp = *head;;

    while( temp->link != NULL){
          temp=temp->link;
         }
          temp->link=newNode;
          newNode->prev= temp;

     cout<<"Now list is : ";
     print(*head);
}
```

- **At a Specific Position**

1. If position = 1, insert at beginning.
2. Traverse to node at position-1.
3. Create new node.
4. Set new node's next = node->next.
5. Set new node's prev = node.
6. If node->next is not NULL, set node->next->prev = new node.
7. Set node->next = new node.

```
void insertPosition(struct dnode **head){
    int pos,newValue;
    cout<<"Enter the position: ";
    cin >> pos;
     if(pos == 1){
        insertBeginning(&(*head));
        return;
    }
    cout<<"Enter new value: ";
    cin >> newValue;

    struct dnode *newNode = (struct dnode*)malloc(sizeof(struct dnode));
    newNode->data=newValue;

    struct dnode *temp = *head;

    for(int i=1;i<pos-1;i++)
        temp=temp->link;

    newNode->link=temp->link;
    newNode->prev=temp;

    if(temp->link)
        temp->link->prev=newNode;

    temp->link=newNode;

    print(*head);
}
```

- **In a Sorted List**

1. Create a new node with value.
2. If list empty or head's value > new value, insert at beginning.
3. Else, traverse to first node with value > new value.
4. Insert new node before that node:
    - new node's next = node
    - new node's prev = node->prev
    - node->prev->next = new node
    - node->prev = new node

```
void insertSorted(struct dnode **head){
     int newValue;
      cout<<"Enter new value: ";
      cin >> newValue;
    struct dnode *newNode=(struct dnode*)malloc(sizeof(struct dnode));
    newNode->data=newValue;

    struct dnode *temp=*head;

    if(temp==NULL || newValue < temp->data){
        newNode->link=temp;
        newNode->prev=NULL;
        if(temp) temp->prev=newNode;
        *head=newNode;
        return;
    }

    while(temp->link && temp->link->data<newValue)
        temp=temp->link;

    newNode->link=temp->link;
    if(temp->link) temp->link->prev=newNode;
    temp->link=newNode;
    newNode->prev=temp;

    cout<<"List is : ";
    print(*head);
}
```

(b) Delete a node from –

• Beginning
• End
• At a specific location
• All occurrence of nodes with a specific value

- **From Beginning**

1. If head is NULL, list empty.
2. Set temp = head.
3. Update head = head->next.
4. If head not NULL, head->prev = NULL.
5. Delete temp.

```
void deleteBeginning(struct dnode **head){
    if(*head == NULL){
        cout << "List is empty. Nothing to delete."<<endl;
        return;
    }

    struct dnode *temp = *head;
    *head = (*head)->link;

    if(*head)
        (*head)->prev = NULL;

    free(temp);
    cout << "Now list is : ";
    print(*head);
}
```

- **2. From End**

1. If head is NULL, list empty.
2. Traverse to last node temp.

3. Set temp->prev->next = NULL.
4. Delete temp.

```
void deleteEnd(struct dnode **head){
    if(*head == NULL){
        cout << "List is empty. Nothing to delete.\n";
        return;
    }

    struct dnode *temp = *head;

    while(temp->link != NULL)
        temp = temp->link;

    if(temp->prev)
        temp->prev->link = NULL;
    else
        *head = NULL;

    free(temp);
    cout << "Now list is : ";
    print(*head);
}
```

- **3. From Specific Position**

1. If position = 1, delete from beginning.
2. Traverse to node at position temp.
3. Set temp->prev->next = temp->next.
4. If temp->next not NULL, set temp->next->prev = temp->prev.
5. Delete temp.

```
void deletePosition(struct dnode **head){
    int pos;
    cout << "Enter the position: ";
    cin >> pos;

    if(*head == NULL){
        cout << "List is empty. Nothing to delete.\n";
        return;
    }

    struct dnode *temp = *head;

    if(pos == 1){
        deleteBeginning(head);
        return;
    }

    for(int i = 1; i < pos && temp != NULL; i++)
        temp = temp->link;

    if(temp == NULL){
        cout << "Position out of range.\n";
        return;
    }
```

- **Delete All Occurrences of a Value**

1. Traverse the list.
2. For each node with matching value:
    - Update prev->next = next
    - If next not NULL, next->prev = prev
    - Delete node

```
void deleteAllOccurrences(struct dnode **head){
    int val;
    cout << "Enter value to delete all occurrences: ";
    cin >> val;

    if(*head == NULL){
        cout << "List is empty. Nothing to delete.\n";
        return;
    }

    struct dnode *temp = *head;

    while(temp != NULL){
        if(temp->data == val){
            struct dnode *toDelete = temp;

            if(temp->prev)
                temp->prev->link = temp->link;
            else
                *head = temp->link;  // Node is head

            if(temp->link)
                temp->link->prev = temp->prev;

            temp = temp->link;
            free(toDelete);
        } else {
            temp = temp->link;
        }
    }

    cout << "Now list is : ";
    print(*head);
}
```

(c) Update a node with new value

1. Start from the head of the list.
2. Traverse the list node by node.
3. Compare each node's data with the old value.
4. If a match is found, replace node's data with the new value.
5. Stop if updating only the first occurrence, otherwise continue traversal.
6. If traversal ends and no match is found, display "Value not found".

```
void update(struct dnode **head){
      int old,newVal;
      cout<<"Enter old data : ";
      cin >> old;
      cout<<"Enter new data : ";
      cin >> newVal;
      struct dnode *temp = *head;
     while(temp != NULL){
         if(temp->data==old)
             temp->data=newVal;
         temp=temp->link;
     }
     cout<<"Now list is : ";
     print(*head);
}
```

(d) Copying a linked list to another linked list

1. Initialize new list head as NULL.
2. Traverse the original list from head.
3. For each node in the original list:
   a. Create a new node.
   b. Copy the data into the new node.
   c. Set new node's next to NULL.
   d. Insert the new node at the end of the new list.
4. Continue until the original list ends.
5. The new list is now a copy of the original list.

```
void copyList(struct dnode* head){
    struct dnode *newHead=NULL,*tail=NULL;

    while(head){
        struct dnode *newnode=(struct dnode*)malloc(sizeof(struct dnode));
        newnode->data=head->data;
        newnode->link=NULL;

        if(!newHead){
            newHead=tail=newnode;
            newnode->prev=NULL;
        }else{
            tail->link=newnode;
            newnode->prev=tail;
            tail=newnode;
        }
        head=head->link;
    }
    cout<<"Coppy : ";
    print(newHead);
}
```

(e) Merging two linked lists

1. If first list is NULL, return second list.
2. If second list is NULL, return first list.
3. Traverse to the last node of the first list.
4. Set last node's next pointer to head of second list.
5. Set second list head's prev pointer to last node of first list.
6. The merged list head is the head of the first list.

```
void mergelist(struct dnode* old){
    cout<<"Enter another list ";
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    struct dnode *newHead = NULL, *temp, *current;

    cout << "Enter elements: ";
    for(int i = 0; i < n; i++){
        current = (struct dnode*)malloc(sizeof(struct dnode));
        cin >> current->data;

        current->prev = NULL;
        current->link = NULL;

        if(newHead == NULL){
            newHead = current;
            temp = current;
        }
        else{
            temp->link = current;
            current->prev = temp;
            temp = current;
        }
    }
    cout <<"The 1st list is : ";
    print(old);
    cout<<"The 2nd list is : ";
    print(newHead);


    struct dnode *tempo=old;

    while(tempo->link)
        tempo=tempo->link;

    tempo->link=newHead;
    newHead->prev=tempo;

    cout<<"After merge two list : ";
    print(old);
}
```

(f) Splitting a linked list

1. Initialize two pointers slow and fast to head.
2. Move slow one node at a time.
3. Move fast two nodes at a time.
4. Continue until fast reaches the end of the list.
5. slow will point to the middle node.
6. Set first list head = head.
7. Set second list head = slow->next.
8. Break the list by setting slow->next = NULL.
9. If second list exists, set second list head's prev = NULL.
10. Now two separate doubly linked lists are formed.

```
void split(dnode* head) {
    if (!head) return;

    dnode *first = NULL, *second = NULL;
    dnode *slow = head, *fast = head;

    while (fast->link && fast->link->link) {
        slow = slow->link;
        fast = fast->link->link;
    }

    first = head;
    second = slow->link;

    slow->link = nullptr;
    if (second)
        second->prev = nullptr;

    cout << "First half: ";
    print(first);
    cout << "Second half: ";
    print(second);
}
```

4.Time Complexity Analysis

Insertion – Beginning/End is O(1) as pointers are adjusted directly; specific/sorted needs traversal, so **O(N)**. Space is always **O(1)**.

Deletion – Beginning/End is O(1) for direct pointer updates; specific/all occurrences require scanning, **O(N)**, **O(1)** space as nodes are unlinked in-place.

Update node – Traverse nodes to change values; **O(N)** time because each node may be checked, **O(1)** space.

Copy list – Traverse original and create new nodes; **O(N)** time as every node is copied, **O(N)** space for new list.

Merge lists – Traverse both lists to link them; **O(N+M)** time as each node is touched once, **O(1)** in-place or **O(N+M)** if creating a new list.

Split list – Traverse to split point and adjust pointers; **O(N)** time to reach the split point, **O(1)** space since nodes are just relinked.